

AULA 05 • MÓDULO ESPECÍFICO PROFISSIONAL

Estruturas de Repetição

Como ensinar o computador a repetir tarefas sem reescrever código — contadores, acumuladores e os três tipos de laço.



DURAÇÃO
4 horas



PROFESSOR
ALEXANDRE SANTANA



DATA
04/07/2026

RECAPITULANDO E AVANÇANDO

Onde paramos & para onde vamos



AULA ANTERIOR

O que vimos na Aula 04

- ✓ Condicional simples (se / if)
- ✓ Se-senão (if / else)
- ✓ Condicionais encadeadas (else if)
- ✓ Estrutura switch / caso
- ✓ Operador ternário

HOJE VAMOS APRENDER

Objetivos da Aula

- Entender o conceito de laço (loop)
- Usar o enquanto / while
- Usar o para / for com contador
- Compreender o repita / do-while
- Reconhecer erros de loop infinito

ROTEIRO DE HOJE

Cronograma da Aula (4h)



1º BLOCO

50 min

Conceito de Laço

O que é repetição e contador



2º BLOCO

55 min

Enquanto (while)

Laço baseado em condição



3º BLOCO

55 min

Para (for)

Laço baseado em contador



4º BLOCO

60 min

Repita + Desafios

do-while e loops aninhados

O que é uma Estrutura de Repetição?

Definição

É uma estrutura que faz o computador executar um bloco de comandos várias vezes, enquanto uma condição for verdadeira. É a tradução em código do conceito de "repita até acabar".

Contador

Variável que controla quantas vezes repetiu

Condição

Define quando o laço deve parar

Corpo

Bloco de comandos que se repete

Exemplo: contar de 1 a 5

Algoritmo "contar"

Var

i: inteiro

Início

i <- 1

enquanto (i <= 5) faça

 escreva(i)

 i <- i + 1

fimenquanto

FimAlgoritmo

Os 3 Tipos de Laço

Cada um resolve um tipo de situação diferente — escolha o certo e seu código fica mais limpo:

while	for	do	nested	break
enquanto	para	repita	aninhado	break
Repete enquanto condição for verdadeira	Repete um número fixo de vezes	Executa ao menos uma vez	Laço dentro de laço	Interrompe o laço antes do fim
while	for	do-while	nested	break

while vs for vs do-while

Os três fazem a mesma coisa (repetir), mas cada um é ideal para uma situação diferente.

while (enquanto)

Pré-teste	Verifica a condição antes de executar
Flexível	Ideal quando não se sabe quantas vezes
Zero voltas	Pode nunca executar o corpo
Exemplo	leitura de dados até digitar "sair"

for (para) / do-while (repita)

Contador	for é ideal quando sabe o total de vezes
Pós-teste	do-while executa ao menos uma vez
Menu	do-while perfeito pra menus de sistema
Início	for garante execução exata de N vezes

Laços: Faça vs Evite

✓ FAÇA ✓

i <- 1; enquanto i <= 10

Inicialize o contador antes do laço

i <- i + 1 dentro do corpo

Sempre incremente dentro do laço

para i de 1 ate N

Use for quando o total é conhecido

Use variável descritiva: contador

Nome que indica a função dela

Teste a condição de parada

Defina claramente o critério de fim

! EVITE X

enquanto (verdadeiro)

Loop infinito sem condição de saída

Esquecer o i <- i + 1

Contador que nunca muda = loop infinito

Usar i antes de inicializar

Variável não inicializada = erro

Modificar o contador de for manualmente

Confunde quem lê o código

Laços aninhados sem necessidade

Problema simples não precisa de aninhamento

Enquanto / While no VisuAlg (55 min)



Abra o VisuAlg e implemente cada exercício — siga a ordem: pseudocódigo → teste → ajuste

PASSO A PASSO

- 1 Declare o contador antes do enquanto
- 2 Coloque a condição de parada certa
Ex.: enquanto (i <= 10) faça
- 3 Incremente o contador dentro do corpo
- 4 Execute com F9 e observe no painel
- 5 Teste com diferentes valores de início

O QUE OBSERVAR

Pontos importantes

- ✓ Sem incremento = loop infinito
- ✓ Condição errada = loop não executa
- ✓ Painel mostra variável em tempo real
- ✓ Use **Ctrl+F2** para parar loop infinito

Exercício guiado — Soma de N números



Como construir

Receba N e some os N números seguintes:

- ✓ Declare: `soma <- 0, i <- 1`
- ✓ enquanto (`i <= n`) faça
- ✓ `leia(numero)`
- ✓ `soma <- soma + numero`

🕒 Tempo: 25 minutos para concluir



Dicas importantes

Atenção a estes pontos:



Acumulador

soma começa em 0 — nunca em lixo



Contador

i começa em 1 e vai até n



✓ Teste com N = 5

Digite 10, 20, 30, 40, 50. Soma = 150

Os 3 Padrões Mais Usados de Laço



Noventa por cento dos laços que você escreverá na vida profissional se encaixam em um destes três padrões. Reconheça-os. Todo conteúdo criado (textos, imagens, vídeos, código, design) tem um autor e é protegido por lei. Usar sem autorização é violação — mesmo na internet.

CONTAGEM SIMPLES

- Contar de A até B
- para i de 1 ate 10
- Resultado: imprime 1, 2, 3... 10
- Use para tabuadas, listas e tabelas
- Mais comum com for



ACUMULADOR

- Somar ou multiplicar numa sequência
- `soma <- soma + numero`
- Resultado: soma total de N valores
- Variável extra que "guarda" o total



BUSCA

- Percorrer até encontrar
- se valor == alvo entao parar
- Resultado: posição ou confirmação
- Combine com if e break

Loops Aninhados



Atenção

Loop Aninhado

Laço dentro de laço — use com cuidado

Como funciona

Um loop aninhado executa o loop interno completamente para cada volta do loop externo. Para cada valor de *i*, o *j* percorre todo o seu ciclo.

Estrutura clássica:



1. **Loop externo** para *i* de 1 ate 3



2. **Loop interno** para *j* de 1 ate 3



3. **Corpo** escreva(*i*, " x ", *j*)



4. **Total** $3 \times 3 = 9$ execuções do corpo

4 Regras para Laços sem Bugs

Checklist obrigatório • Aplique *sempre* estas 4 regras antes de dar um laço como pronto. A maioria dos bugs de laço vem de violar uma delas.

As 4 regras do laço correto



Inicialize antes

Todo contador ou acumulador precisa de valor antes do laço



Condição alcançável

A condição de parada deve ser atingível com o incremento usado



Incremento garantido

O contador deve mudar em cada volta — sem exceção



Teste de mesa

Execute o laço na cabeça linha a linha antes de rodar

Erros Comuns em Laços

Os 3 erros que mais causam loop infinito e travamento do programa:



LOOP INFINITO

Esquecer o incremento do contador — o laço nunca atinge a condição de parada.

SEMPRE INCREMENTE



OFF BY ONE

Usar < em vez de <= (ou vice-versa) — erra por 1 a quantidade de execuções.

VERIFIQUE O SINAL



VARIÁVEL NÃO INICIALIZADA

Usar o acumulador sem zerar antes — resultado começa com "lixo".

SEMPRE INICIALIZE

Quando Usar Cada Tipo de Laço



A escolha certa economiza código. Aprenda a reconhecer o padrão do problema antes de escolher o laço.



01

Use WHILE quando não sabe quantas vezes

IDEAL PARA

- ✓ Leitura até digitar "sair"
- ✓ Validação: repetir até resposta válida
- ✓ Processar arquivo até o fim
- ✓ Número de repetições indefinido

EVITE

Quando o total de repetições é conhecido



02

Use FOR quando sabe o total de vezes

IDEAL PARA

- ✓ Tabuada do 1 ao 10
- ✓ Imprimir lista de N itens
- ✓ Percorrer todos os elementos

EVITE

Quando a condição de parada é dinâmica



03

Use DO-WHILE para executar ao menos 1x

IDEAL PARA

- ✓ Menu de opções de sistema
- ✓ Pedir dado e validar depois

EVITE

Quando o corpo pode não executar nunca

Desafios com Repetição (60 min)



Escolha 3 desafios

Implemente no VisuAlg:

1

Tabuada interativa

Receba um número e mostre a tabuada de 1 a 10 com for

2

Adivinhe o número

Gere 42 e peça palpites com while até acertar

3

Contagem regressiva

Receba N e conte de N até 1 mostrando cada número



ENTREGA

45 min para fazer + 15 min apresentar

Como entregar:

Salve cada .alg no Google Drive da turma. Inclua um print do programa rodando com 3 entradas diferentes.

O professor circula pela sala dando feedback enquanto você programa.

PRÓXIMA AULA • AULA 06

Vetores e Matrizes

- O que é um vetor (array)
- Declarar e percorrer com for
- Matrizes bidimensionais
- Busca e ordenação simples
- Estruturas de dados homogêneas

Excelente aula! Pratique os laços em casa. 🚀